COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures & Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	09/15/2026	**DATE SUBMITTED:**	09/15/2026
NAME:	Baltazar, Kenji	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 **Hands-on Activity 7.1 Sorting Algorithms Pt1**

1. PROCEDURE OUTPUT

CODE + OUTPUT

```
1   #include <iostream>
2   #include <cstdlib>
3   #include <time.h>
4   #include "sortingAlgo2.h"
5
6   void display(int arr[]);
7   const int maxSize = 100;
8
9   int main() {
10      int unsortedArr[maxSize];
11      srand(time(0));
12      for (int i = 0; i < maxSize; i++) {
13          unsortedArr[i] = rand() % 100 + 1;
14      }
15      std::cout<<"random numbers: \n";
16      display(unsortedArr);
```

```
33  void display(int arr[]) {
34      for (int i = 0; i < maxSize; i++) {
35          std::cout << arr[i] << " ";
36      }
37      std::cout << std::endl;
38  }
```

```
random numbers:
4 88 88 38 58 26 87 24 71 94 74 33 17 14 25 57 39 62 19 32 3 77 5 31 1 43 44 97 67 37 88 96 34 61 83
46 97 42 71 9 29 42 52 19 74 58 34 81 72 17 33 82 19 49 78 91 27 3 66 66 82 1 28 7 56 93 27 45 50 51
27 37 90 39 9 8 50 100 30 46 33 64 65 31 21 14 64 57 63 32 6 38 85 48 39 60 98 89 8 30
```

OBSERVATION

This is the main function where every sorting algorithm is called, the main function starts of by using a random number generator as seen on line 11. Then uses a loop called display which is found in line 33, where its purpose is to display up to maxSize random numbers which in this case is 100. Having an O(n) worst & best time complexity. This is mainly from HOA7.

Table 8-1. Array of Values for Sort Algorithm Testing

CODE + OUTPUT

```
10  template <typename T>
11  void shellSort(T arr[], int size){
12      int interval = size / 2;
13      while (interval >= 1) {
14          for (int i = interval; i < size; i++) {
15              T temp = arr[i];
16              int j = i;
17              while (j >= interval && arr[j - interval] > temp) {
18                  arr[j] = arr[j - interval];
19                  j = j - interval;
20              }
21              arr[j] = temp;
22          }
23          interval = interval / 2;
24      }
25  }
```

```
Shell Sort:
1 1 3 3 4 5 6 7 8 8 9 9 14 14 17 17 19 19 19 21 24 25 26 27 27 27 28 29 30 30 31 31 32 32 33 33 33 34
34 37 37 38 38 39 39 39 42 42 43 44 45 46 46 48 49 50 50 51 52 56 57 57 58 58 60 61 62 63 64 64 65 66
66 67 71 71 72 74 74 77 78 81 82 82 83 85 87 88 88 88 89 90 91 93 94 96 97 97 98 100
```

OBSERVATION

This sorting algo is the shell sort, where it first sets the **interval = size/2**, then the for loop checks all the elements that depends on interval positions, whilst the inner loop moves those larger values than the rest to the right until temp will place them in their rightful positions. This is a more enhanced version than insertion sort as it uses gaps, though it is more unstable in a sense. It also has a time complexity of $O(n^2)$.

Table 8-2. Shell Sort Technique

CODE + OUTPUT

```
38  template <typename T>
39  void merge(T array[],int left,int middle,int right,int maxSize){
40      T temp[maxSize];
41      int i = left;
42      int j = middle + 1;
43      int k = left;
44      while(i <= middle && j <= right){
45          if (array[i] < array[j]) {
46              temp[k] = array[i];
47              i++;
48          }else {
49              temp[k] = array[j];
50              j++;
51          }
52          k++;
53      }
54      while(i <= middle){temp[k] = array[i]; i++; k++;}
55      while(j <= right){temp[k] = array[j]; j++; k++;}
56
57      for(int x = left; x <= right; x++){
58          array[x] = temp[x];
59      }
60  }
```

```
62    template <typename T>
63    void mergeSort(T array[],int left,int right,int maxSize){
64        if(left < right) {
65            int middle = (left + right) / 2;
66            mergeSort(array, left, middle, maxSize);
67            mergeSort(array, middle + 1, right, maxSize);
68            merge(array, left, middle, right, maxSize);
69        }
70    }
```

```
Merge Sort:
1 1 3 3 4 5 6 7 8 8 9 9 14 14 17 17 19 19 19 21 24 25 26 27 27 27 28 29 30 30 31 31 32 32 33 33 33 34
34 37 37 38 38 39 39 39 42 42 43 44 45 46 46 48 49 50 50 51 52 56 57 57 58 58 60 61 62 63 64 64 65 66
66 67 71 71 72 74 74 77 78 81 82 82 83 85 87 88 88 88 89 90 91 93 94 96 97 97 98 100
```

OBSERVATION

This sorting algorithm is the merge sort, where the array is first divided into 2 parts. Then the 2 mergeSort calls sort the left and right part, the the merge function is the one that compares both parts then we put them in temp again to put them in their right place later on. As this is also halving and halving the data set, this is from O(log n) but because it has loops, it runs in n times so O(n log n)

Table 8-3. Merge Sort Algorithm

CODE + OUTPUT

```cpp
template <typename T>
int partition(T array[], int low, int high) {
    T pivotValue = array[high];
    int position = low;

    for(int i = low; i < high; i++){
        if (array[i] < pivotValue) {
            std::swap(array[i], array[position]);
            position++;
        }
    }
    std::swap(array[position], array[high]);

    return position;
}

template <typename T>
void quickSort(T array[], int low, int high) {
    if(low < high){
        int pivot = partition(array, low, high);

        quickSort(array, low, pivot - 1);
        quickSort(array, pivot + 1, high);
    }
}
```

```
Quick Sort:
1 1 3 3 4 5 6 7 8 8 9 9 14 14 17 17 19 19 19 21 24 25 26 27 27 27 28 29 30 30 31 31 32 32 33 33 33 34
34 37 37 38 38 39 39 39 42 42 43 44 45 46 46 48 49 50 50 51 52 56 57 57 58 58 60 61 62 63 64 64 65 66
66 67 71 71 72 74 74 77 78 81 82 82 83 85 87 88 88 88 89 90 91 93 94 96 97 97 98 100
```

OBSERVATION

This is the Quick sort algo, where it first makes the last element the pivot. The for loop then checks the values to compare them with the pivot and then swaps smaller values towards it, then placed in its proper position. Then it could now have both left and right partitions which the checking goes on and on until everything is sorted. This is O(n log n) as it keeps partitioning.

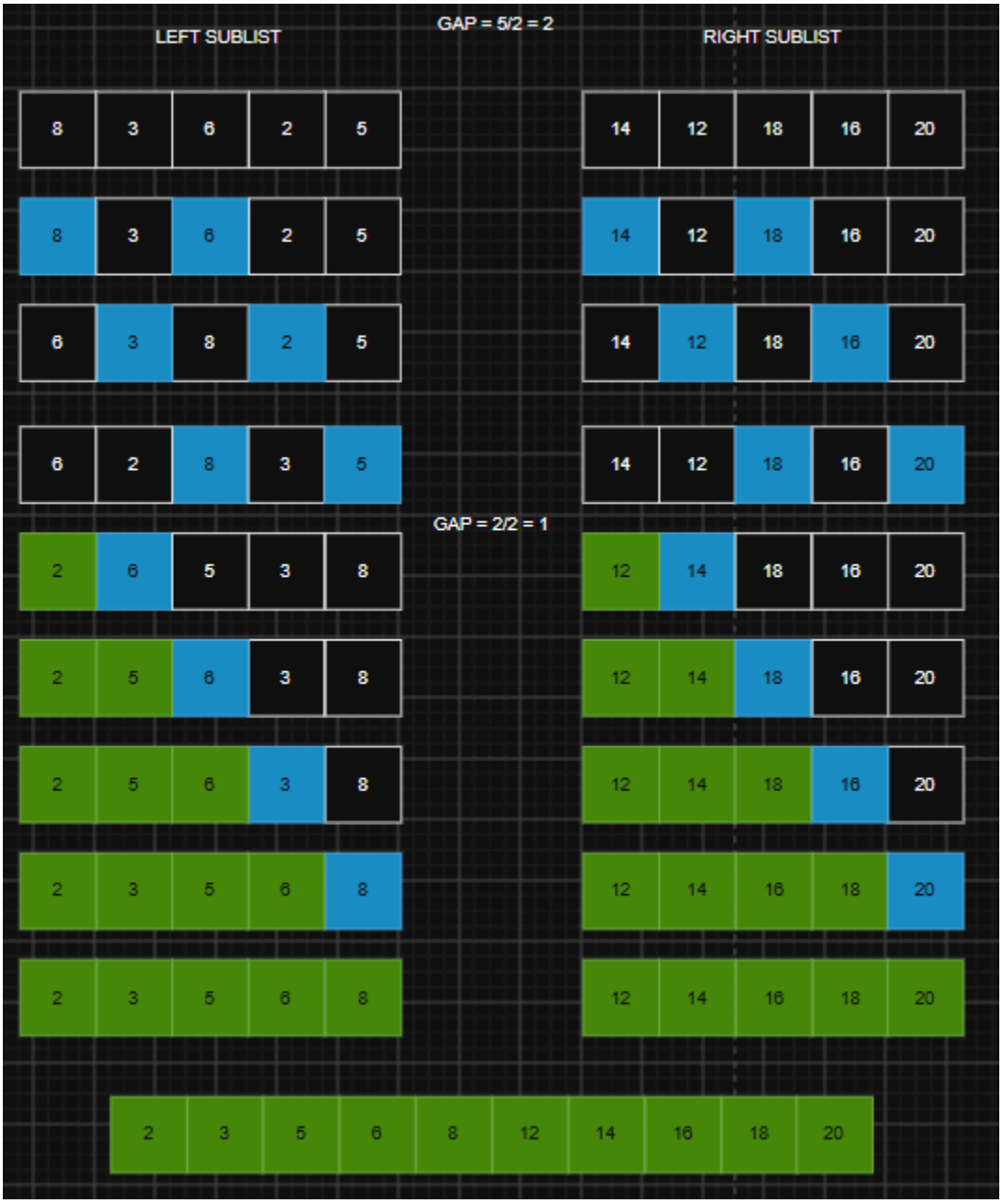
Table 8-4. Quick Sort Algorithm

2. ANSWER TO SUPPLEMENTARY ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

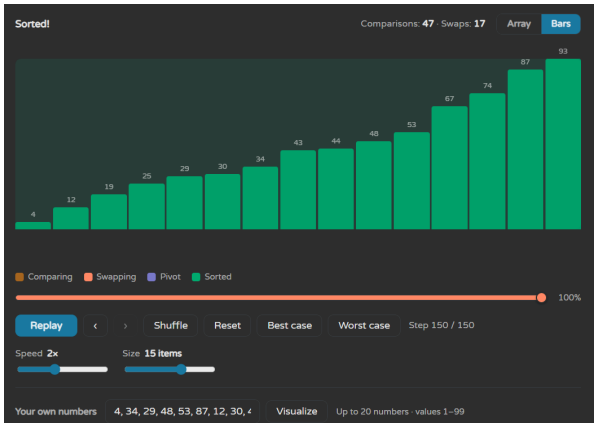
Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? *Demonstrate an example.*

- Yes, we can definitely sort the left and right partitions using other algo aside from quick sort. As we can see in this given example, after the quick sort partitions left and right, we use the shell sort to sort out both partitions.



Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- I would choose quick sort as it is generally have the O(n log n) as its most average time complexity, if we compare it to merge sort which has a worst case than quick sort when it keeps on comparing gaps but it is still not enough so another long insertion sort will happen. Also, both of them have O(n log n) because they both can walk through log n (dividing depending on n) while checking every n times. As seen below quick sort only need 47 comparison and 17 swap while merge needs 42 comparison and 59 swaps.

QUICK	MERGE
	

3. CONCLUSION

In this activity, I was able to learn another 3 sorting algorithm which is more expert than the first 3 sorting algorithms. I first learn about shell sort which is a more advance level than insertion sort, this sorting algorithm uses gaps to compare values and swaps them if the left side is greater than the right side, until it ends up with gap 1 which goes back to being an insertion sort. Next is the merge sort which separates the array into tiny arrays and splitting them up until there are only 2 element left, then those 2 element will be compared to one another then compared to another 2 element that has been compared to one another then those will be compared again to another array until they all merge back together in a sorted form. Lastly, I learned about quick sort which uses a pivot and uses it to compare against the different elements in the array until there it is back to where it is supposed to be leaving partitions, then another pivot will be chosen again and this will go on until all is sorted out. Overall, I understand the basic concepts and general idea of these 3 and that there are some downsides as well for them even though they are a much advanced version like shell sort uses gaps so it is more unstable than insertion sort but all of these 3 are much better when there are big data involved.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.